

Day 11: Classes and Objects

- Class structure
- Object creation
- Instance variables
- Methods

Practice:

- Student class
- Employee class

Before Learning Class and Object

Mundhu oka question.

Computer ki ela telustundi?

Example:

You want to store details of a student.

Student Name

Student Age

Student Marks

One student ki variables create cheyyachu.

```
String name="Ravi";
```

```
int age=20;
```

```
int marks=95;
```

But college lo 1000 students unnaru.

Appudu?

```
String name1;
```

```
String name2;
```

```
String name3;
```

```
String name4;
```

...

Ila chesthava?

Impossible.

Programming lo repeated data ni easy ga handle cheyyadaniki **Classes and Objects** introduce chesaru.

Real Life Example

Imagine a Car Factory.

Factory first car ni create cheyyadu.

First...

Car Design create chestundi.

Aa design lo untayi.

- Engine
- Wheels
- Doors
- Colour
- Steering

Ee design ni use chesi thousands of cars create chestaru.

Design = **Class**

Real Car = **Object**

Another Example

House Blueprint

Architect blueprint draw chestadu.

Blueprint lo rooms untayi.

Kitchen untundi.

Hall untundi.

Bedroom untundi.

Kani blueprint lo manam stay cheyyalem.

Blueprint ni use chesi house build chestaru.

Blueprint = Class

House = Object

Human Example

Doctor Hospital lo birth ayina babies.

Human structure same.

- 2 Eyes
- 2 Legs
- Name
- Age

Structure same.

Person different.

Human structure = Class

Every person = Object

Definition of Class

Simple Definition

Class ante

Object ni create cheyyadaniki blueprint.

OR

Template.

OR

Design.

OR

Skeleton.

Technical Definition

Class is a user-defined data type that contains

- Variables
- Methods

- Constructors
 - Objects ni create cheyyadaniki template
-

Why Class?

Suppose College Management System.

Need details like

Student

Teacher

Employee

Principal

Library

Books

Hostel

Fees

Veetannitiki separate blueprint kavali.

Anduke Classes.

Class Structure

Syntax:

```
class Student
```

```
{
```

```
}
```

Simple ga empty class.

Now variables add cheddham.

```
class Student
```

```
{
```

```
    String name;
```

```
    int age;  
    int marks;  
}
```

Student blueprint ready.

But...

Student ekkada?

Ledu.

Only design undi.

Important Point

Class create chesthe

Object create avvadu.

Blueprint matrame create avutundi.

Example

Blueprint ✓

House ✗

Exactly same.

Class ✓

Object ✗

Object

Definition:

Object is an instance of a class.

Easy definition:

Object ante

Real implementation of class.

Example

Blueprint

↓

House

Class

↓

Object

Creating Object

Syntax

```
Student s1 = new Student();
```

Breakdown:

Student

Datatype.

Student

Means

"I am creating Student type variable."

s1

Variable name.

Student s1

Means

Student type reference variable.

new

Very important keyword.

new ante

"Memory allocate cheyyi."

Student()

Constructor call.

(Constructors next lesson lo complete ga nerchukuntam.)

Complete

Student s1=new Student();

Means

Create one Student object in memory.

Memory Representation

Suppose

Student s1=new Student();

Memory

Heap

```
+-----+
| name = null |
| age =0      |
| marks=0     |
+-----+
```

▲
|
s1

s1 object ni point chestundi.

Why new keyword?

Question:

Student s1;

Enough aa?

No.

Why?

Only reference variable create ayyindi.

Object create kaledu.

Memory allocate kaledu.

Need

new Student();

Appude object create avutundi.

Difference

Without new

Student s1;

Memory

s1

↓

Nothing

With new

Student s1=new Student();

Memory

s1

↓

Object

Accessing Variables

s1.name="Rahul";

Dot operator use chestam.

Dot ante

Object members access cheyyadaniki.

Example

s1.age=20;

s1.marks=90;

Now object

Name=Rahul

Age=20

Marks=90

Print

System.out.println(s1.name);

System.out.println(s1.age);

System.out.println(s1.marks);

Output

Rahul

20

90

Multiple Objects

One class.

Many objects.

```
Student s1=new Student();
```

```
Student s2=new Student();
```

```
Student s3=new Student();
```

Memory

Student

↓

s1

↓

Student Object

s2

↓

Student Object

s3

↓

Student Object

Every object separate.

Example

s1.name="A";

s2.name="B";

s3.name="C";

Independent.

Instance Variables

Now important topic.

Example

class Student

{

String name;

int age;

}

name

age

Ee variables ni

Instance Variables antaru.

Why Instance Variables?

Because

Every object ki separate copy untundi.

Example

s1

name=Rahul

age=20

s2

name=Anu

age=19

Different values.

Memory

s1

↓

name=Rahul

age=20

s2

↓

name=Anu

age=19

Default Values

Object create chesina ventane.

Java automatic ga values assign chestundi.

Datatype Default

int	0
double	0.0
float	0.0
boolean	false
char	'\u0000'
String	null

Example

```
Student s1=new Student();
```

```
Immediately
```

```
name=null
```

```
age=0
```

Instance Variable Lifetime

Object create ayyaka start.

Object destroy ayye varaku untundi.

Methods

Already methods nerchukunnam.

Class lo methods create chestam.

Example

```
class Student
```

```
{
```

```
    String name;
```

```
    void display()
```

```
{
```

```
        System.out.println(name);
    }
}
```

display() method

Student object details print chestundi.

Call

```
Student s1=new Student();
```

```
s1.name="Rahul";
```

```
s1.display();
```

Output

Rahul

Complete Flow

Step 1

Create class.

↓

Step 2

Declare variables.

↓

Step 3

Write methods.

↓

Step 4

Create object.

↓

Step 5

Assign values.



Step 6

Call methods.

Example Program

```
class Student
```

```
{
```

```
    String name;
```

```
    int age;
```

```
    void display()
```

```
    {
```

```
        System.out.println(name);
```

```
        System.out.println(age);
```

```
    }
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        Student s1=new Student();
```

```
        s1.name="Rahul";
```

```
        s1.age=20;
```

```
        s1.display();
```

```
    }
```

```
}
```

Output

Rahul

20

How JVM Executes This Program

Suppose code:

```
Student s1 = new Student();
```

```
s1.name = "Rahul";
```

```
s1.display();
```

Step 1

JVM loads Student class.

Class loaded ✓

Step 2

new Student() execute chestundi.

Heap memory lo object create chestundi.

Heap

```
name=null
```

```
age=0
```

Step 3

Reference variable s1 aa object address ni store chestundi.

```
s1
```


↓

Object

Step 4

s1.name="Rahul";

Object value change.

name=Rahul

Step 5

display()

Method execute.

Object values print chestundi.

Dot Operator

Dot (.) is used to access members (variables and methods) of an object.

s1.name

Access variable.

s1.display();

Call method.

Why can't we write directly?

Wrong

name="Rahul";

Compiler asks:

"Whose name?"

Need object.

Correct

s1.name="Rahul";

Why Multiple Objects?

One blueprint.

Many students.

Student

↓

Object1

Object2

Object3

Object4

Each object stores its own data.

Student Class Example

```
class Student
```

```
{
```

```
    String name;
```

```
    int age;
```

```
    int marks;
```

```
    void display()
```

```
{
```

```
        System.out.println("Name : "+name);
```

```
        System.out.println("Age : "+age);
```

```
        System.out.println("Marks : "+marks);
```

```
    }  
}  
  
public class Main  
{  
    public static void main(String args[])  
    {  
        Student s1=new Student();  
  
        s1.name="Asha";  
        s1.age=20;  
        s1.marks=95;  
  
        s1.display();  
  
        Student s2=new Student();  
  
        s2.name="Vikram";  
        s2.age=21;  
        s2.marks=88;  
  
        s2.display();  
    }  
}
```

Output:

Name : Asha

Age : 20

Marks : 95

Name : Vikram

Age : 21

Marks : 88

Notice that s1 and s2 are different objects with different values.

Employee Class Example

```
class Employee
```

```
{
```

```
    int id;
```

```
    String name;
```

```
    double salary;
```

```
    void display()
```

```
    {
```

```
        System.out.println("ID : "+id);
```

```
        System.out.println("Name : "+name);
```

```
        System.out.println("Salary : "+salary);
```

```
    }
```

```
}
```

```
public class Main
```

```
{
```

```
    public static void main(String args[])
```

```
    {
```

```
        Employee e1=new Employee();
```

```
        e1.id=101;
```

```
        e1.name="Kiran";
```

```
        e1.salary=50000;
```

```
        e1.display();
    }
}
```

Output

ID : 101

Name : Kiran

Salary : 50000.0

Class vs Object

Class	Object
Blueprint	Real thing
Template	Instance of class
No memory for individual data	Occupies memory
Defined once	Can create many
Logical design	Physical entity

Example:

Class = Student

Objects = s1, s2, s3

Instance Variables vs Local Variables

Instance Variable	Local Variable
Declared inside class, outside methods	Declared inside a method
Belongs to object	Belongs only to that method

Instance Variable	Local Variable
Gets default values	No default values; must be initialized before use
Lives as long as the object exists	Lives only while the method is executing

Example:

```
class Student
{
    int age; // Instance variable

    void show()
    {
        int x = 10; // Local variable
        System.out.println(age);
        System.out.println(x);
    }
}
```

Common Beginner Mistakes

1. Forgetting new

```
Student s1;
```

```
s1.name = "A";
```

✗ Error (s1 doesn't refer to any object)

Correct:

```
Student s1 = new Student();
```

```
s1.name = "A";
```

2. Calling a method without an object

Wrong:

```
display();
```

Correct:

```
s1.display();
```

3. Assuming one object's data affects another

```
Student s1 = new Student();
```

```
Student s2 = new Student();
```

```
s1.name = "A";
```

```
s2.name = "B";
```

Changing s1.name will **not** change s2.name. They are separate objects.

Interview Questions

1. What is a class?

A class is a blueprint or template used to create objects.

2. What is an object?

An object is an instance of a class that stores its own data and can use the class's methods.

3. Why do we need objects?

Objects allow us to create multiple independent entities from the same class without duplicating code.

4. What is an instance variable?

A variable declared inside a class but outside any method. Every object gets its own copy.

5. Why do we use new?

The new keyword allocates memory in the heap and creates an object.

6. What does the dot (.) operator do?

It accesses an object's variables and methods.

7. Can one class create many objects?

Yes. One class can be used to create any number of objects.

8. Does a class occupy memory like an object?

A class defines the structure, while each object occupies memory to store its own data.

Practice Programs

1. Create a Student class with name, rollNo, and marks. Create three objects and print their details.
2. Create an Employee class with id, name, and salary. Create two employees and display their information.
3. Create a Car class with brand, model, and price. Create three car objects and print their details.
4. Create a Book class with title, author, and pages. Create two book objects and display them.
5. Create a Mobile class with brand, ram, and storage. Create multiple objects and compare their values by printing them.